```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main()
{
    printf("Runtime Error Practice Program\n");

    int a = 100;
    int b = 0;

    printf("Division Result = %d\n", a / b);
    // Error: Division by zero.
    // Solution: if(b != 0) printf("Division Result = %d\n", a / b);


    int *ptr1 = NULL;

    *ptr1 = 50;
    // Error: NULL pointer dereference.
    // Solution: int x=50; ptr1=&x; *ptr1=50;


    int arr[5];

    arr[10] = 100;
    // Error: Array index out of bounds.
    // Solution: arr[4] = 100;
```

```
21
22      int *ptr2;
23
24      *ptr2 = 200;
        // Error: Uninitialized pointer.
        // Solution: int y; ptr2=&y; *ptr2=200;

25
26      char name[5];
27
28      strcpy(name, "Programming");
        // Error: Buffer overflow.
        // Solution: char name[20]; strcpy(name,"Programming");

29
30      printf("%s\n", name);
31
32      int *ptr3;
33
34      printf("%d\n", *ptr3);
        // Error: Uninitialized pointer dereference.
        // Solution: int z=10; ptr3=&z; printf("%d\n",*ptr3);

35
36      FILE *fp;
37
38      fp = fopen("unknown.txt", "r");
```

```
39
40    fscanf(fp, "%d", &a);
      // Error: fp may be NULL.
      // Solution: if(fp!=NULL) fscanf(fp,"%d",&a);

41
42    fclose(fp);
43
44    char *ptr4 = malloc(5);
45
46    strcpy(ptr4, "Computer Science");
      // Error: Heap buffer overflow.
      // Solution: ptr4 = malloc(20);

47
48    printf("%s\n", ptr4);
49
50    free(ptr4);
51
52    printf("%c\n", ptr4[0]);
      // Error: Use after free.
      // Solution: Access ptr4 before free() or allocate again.

53
54    int *ptr5 = malloc(sizeof(int));
55
56    free(ptr5);
57
```

```
58    free(ptr5);
      // Error: Double free.
      // Solution: Remove second free(ptr5);

59
60    int x = 5;
61
62    int y = 0;
63
64    int result = x % y;
      // Error: Modulus by zero.
      // Solution: if(y!=0) result=x%y;

65
66    printf("%d\n", result);
67
68    char str1[10] = "Hello";
69
70    strcat(str1, "WorldProgramming");
      // Error: Buffer overflow.
      // Solution: char str1[30]="Hello";

71
72    printf("%s\n", str1);
73
74    int *numbers;
75
76    numbers = malloc(3 * sizeof(int));
```

```
77
78    for(int i=0; i<10; i++)
      // Error: Loop exceeds allocated memory.
      // Solution: for(int i=0;i<3;i++)
79    {
80        numbers[i] = i;
81    }
82
83    free(numbers);
84
85    int matrix[3][3];
86
87    matrix[5][5] = 100;
      // Error: Array index out of bounds.
      // Solution: matrix[2][2]=100;

88
89    int *ptr6 = NULL;
90
91    printf("%d\n", ptr6[0]);
      // Error: NULL pointer access.
      // Solution: Allocate memory before access.

92
93    char password[8];
94
95    gets(password);
      // Error: Unsafe function gets().
```

```
    // Solution: fgets(password,sizeof(password),stdin);

96
97      printf("%s\n", password);
98
99      int *ptr7 = malloc(sizeof(int));
100
101      *ptr7 = 500;
102
103      free(ptr7);
104
105      *ptr7 = 1000;
    // Error: Use after free.
    // Solution: Do not access ptr7 after free().

106
107      int age;
108
109      printf("Age = %d\n", age);
    // Error: Uninitialized variable.
    // Solution: int age = 18;

110
111      int index = -1;
112
113      arr[index] = 50;
    // Error: Negative array index.
    // Solution: Use index >= 0.
```

```
114
115    int *ptr8;
116
117    if(*ptr8 > 0)
       // Error: Uninitialized pointer.
       // Solution: Initialize ptr8 before dereferencing.
118    {
119        printf("Positive\n");
120    }
121
122    char *text = NULL;
123
124    printf("%s\n", text);
       // Error: NULL string pointer.
       // Solution: if(text!=NULL) printf("%s\n",text);

125
126    int size = 1000000000;
127
128    int *hugeArray;
129
130    hugeArray = malloc(size * sizeof(int));
131
132    hugeArray[0] = 1;
       // Error: malloc may fail and return NULL.
       // Solution: if(hugeArray!=NULL) hugeArray[0]=1;
```

```
133
134     free(hugeArray);
135
136     int *ptr9 = NULL;
137
138     while(1)
        // Error: Infinite loop.
        // Solution: Add a terminating condition.
139     {
140         ptr9++;
141     }
142
143     printf("End of Program\n");
144
145     return 0;
146 }
```